

```
C:\Test\grinder-3.11-binary>grinder-3.11 java net.grinder.Grinder
2015-10-06 15:14:46,072 INFO agent: The Grinder 3.11
2015-10-06 15:14:46,127 INFO agent: connected to console at localhost/127.0.0.1:6372
2015-10-06 15:14:46,228 INFO agent: waiting for console signal
```

Figure 5: The Grinder console command

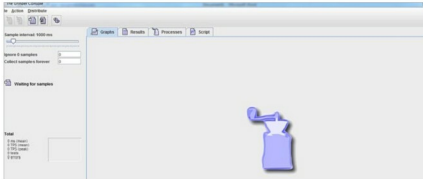


Figure 6: Grinder console

```
C:\Testing\grinder-3.11-binary\grinder-3.11>java net.grinder.Grinder
Error: Could not find or load main class net.grinder.Grinder
```

Figure 7: Error in the running Grinder agent

```
C:\Testing\grinder-3.11-binary\grinder-3.11>java net.grinder.Grinder
2015-10-06 15:14:46,072 INFO agent: The Grinder 3.11
2015-10-06 15:14:46,127 INFO agent: connected to console at localhost/127.0.0.1:6372
2015-10-06 15:14:46,228 INFO agent: waiting for console signal
```

Figure 8: Grinder agent command

```
'Oracle@123'),
    NVPair('submit', 'Login'), ),
    ( NVPair('Content-Type',
'application/x-www-form-urlencoded'),
))

grinder.sleep(28)
request102.GET('/cdacdrml/user/
monitoring/drmHome.html')
```

From the above code, we get a basic idea of the structure of a test script. This script is 98 per cent useful. The script can be parametrised according to need. Right now, I log in with the user name 'oracle', but what if I want to log in with a different user name. To log in with

different names, I need to create an array that stores user names and password combinations. Then, based on The Grinder thread number, I try to log in with the appropriate user name and password.

After changing the parameter's value, the script looks like what follows:

```
credentials = [{"username": "oracle",
"password": "Oracle@123"},

{"username": "postgres",
"password": "Postgres@123"},

{"username": "shweta",
"password": "Shweta1"}];
class TestRunner:
    """A TestRunner instance is created
    for each worker thread."""
```

```
# A method for each recorded page.
def page1(self):
    """POST j_spring_security_check
(requests 101-102)."""
    credential = credentials[grinder.
threadNumber];
    username = credential["username"];
    password = credential["password"];
    # Expecting 302 'Found'
    result = request101.POST('/cdacdrml/
cdacdrml/j_spring_security_check',
    ( NVPair('j_username', username),
    NVPair('j_password', password),
    NVPair('submit', 'Login'), ),
    (NVPair('Content-Type',
'application/x-www-form-urlencoded'),
))

    grinder.sleep(28)
    request102.GET('/cdacdrml/user/
monitoring/drmHome.html')
```

```
return result
```

Running the test

After you have recorded your script, you have two methods to replay it:

1. You can edit the *grinder.properties* file and play the recorded scenario with The Grinder.
2. You can use the console to send your script to an agent. Each agent will need a *grinder.properties* file containing the console address.

Next, copy *grinder-1.py* under your *C:\Testing\grinder-3.11-binary\grinder-3.11\example*

Now, check for the *grinder.properties* file inside *C:\Testing\grinder-3.11-binary\grinder-3.11\example*. If it is found, then edit the existing file; otherwise prepare a new file and put it under The Grinder's example directory. Please see the code below:

```
# Commonly used properties
#
# The file name of the script to run.
#
# Relative paths are evaluated from the
# directory containing the
# properties file. The default is
"grinder.py".
```